



Smart Contract Security Audit Report



The SlowMist Security Team received the DappOSDao team's application for smart contract security audit of the DappOS Staking on 2026.08.20. The following are the details and results of this smart contract security audit:

Token Name :

DappOS Staking

The contract address :

DapposToken.sol: <https://etherscan.io/address/0x951f086a127e280724fd93ccc543f65065afeb5e>

OFTAdapter.sol: <https://etherscan.io/address/0xB0f09ea9ae0515C3551080D4a745C8115aA30e37>

DapposTokenOFT.sol: <https://bscscan.com/address/0xB0f09ea9ae0515C3551080D4a745C8115aA30e37>

Staking.sol: <https://etherscan.io/address/0x37225c21ef84063e76323aa4586212084e366bb9#code>

The audit items and results :

(Other unknown security vulnerabilities are not included in the audit responsibility scope)

NO.	Audit Items	Result
1	Replay Vulnerability	Passed
2	Denial of Service Vulnerability	Passed
3	Race Conditions Vulnerability	Passed
4	Authority Control Vulnerability Audit	Passed
5	Integer Overflow and Underflow Vulnerability	Passed
6	Gas Optimization Audit	Passed
7	Design Logic Audit	Passed
8	Uninitialized Storage Pointers Vulnerability	Passed
9	Arithmetic Accuracy Deviation Vulnerability	Passed

NO.	Audit Items	Result
10	"False top-up" Vulnerability	Passed
11	Malicious Event Log Audit	Passed
12	Scoping and Declarations Audit	Passed
13	Safety Design Audit	Passed
14	Non-privacy/Non-dark Coin Audit	Passed

Audit Result : Passed

Audit Number : 0X002608200001

Audit Date : 2026.08.20 - 2026.08.20

Audit Team : SlowMist Security Team

Summary conclusion : This project includes a staking contract, an ERC20 token contract, an OFT contract, and an OFTAdapter contract. These contracts do not include dark coin functionality. The total supply of contract tokens in the DapposToken and DapposTokenOFT contracts remains constant. Owner can pause and cancel paused the DapposToken and DapposTokenOFT contracts. When paused, tokens cannot be transferred, minted, or destroyed. These contracts don't have the Overflow and the Race Conditions issue.

The source code:

DapposToken.sol

```
// SPDX-License-Identifier: MIT
// Compatible with OpenZeppelin Contracts ^5.0.0
//SlowMist// The contract does not have the Overflow and the Race Conditions issue
pragma solidity ^0.8.22;

import {ERC20} from "@openzeppelin/contracts/token/ERC20/ERC20.sol";
import {ERC20Pausable} from
"@openzeppelin/contracts/token/ERC20/extensions/ERC20Pausable.sol";
import {Ownable} from "@openzeppelin/contracts/access/Ownable.sol";
```

```
contract DapposToken is ERC20, ERC20Pausable, Ownable {
    constructor(
        address _owner,
        address _treasury,
        uint256 _totalSupply
    ) ERC20("DAPPOS", "DOS") Ownable(_owner) {
        require(_treasury != address(0), "Treasury cannot be zero address");
        require(_totalSupply > 0, "Total supply must be greater than zero");

        _mint(_treasury, _totalSupply);
    }

    //SlowMist// Owner can pause and cancel paused the contract
    function pause() public onlyOwner {
        _pause();
    }

    function unpause() public onlyOwner {
        _unpause();
    }

    function _update(
        address from,
        address to,
        uint256 value
    ) internal override(ERC20, ERC20Pausable) {
        super._update(from, to, value);
    }
}
```

DapposTokenOFT.sol

```
// SPDX-License-Identifier: MIT
//SlowMist// The contract does not have the Overflow and the Race Conditions issue
pragma solidity ^0.8.22;

import {Ownable} from "@openzeppelin/contracts/access/Ownable.sol";
import {Pausable} from "@openzeppelin/contracts/utils/Pausable.sol";
import {OFT} from "@layerzerolabs/lz-evm-oapp-v2/contracts/oft/OFT.sol";

contract DapposTokenOFT is OFT, Pausable {
```

```
constructor(
    string memory _name,
    string memory _symbol,
    address _lzEndpoint,
    address _owner
) OFT(_name, _symbol, _lzEndpoint, _owner) Ownable(_owner) {}

//SlowMist// Owner can pause and cancel paused the contract
function pause() public onlyOwner {
    _pause();
}

function unpause() public onlyOwner {
    _unpause();
}

function _update(
    address from,
    address to,
    uint256 value
) internal override whenNotPaused {
    super._update(from, to, value);
}
}
```

DapposTokenAdapter.sol

```
// SPDX-License-Identifier: MIT
//SlowMist// The contract does not have the Overflow and the Race Conditions issue
pragma solidity ^0.8.22;

import {OFTAdapter} from "@layerzerolabs/lz-evm-oapp-
v2/contracts/oft/OFTAdapter.sol";
import {Ownable} from "@openzeppelin/contracts/access/Ownable.sol";

contract DapposTokenAdapter is OFTAdapter {
    constructor(
        address _token,
        address _layerZeroEndpoint,
        address _owner
    ) OFTAdapter(_token, _layerZeroEndpoint, _owner) Ownable(_owner) {}
}
```

Staking.sol

```
// SPDX-License-Identifier: MIT
//SlowMist// The contract does not have the Overflow and the Race Conditions issue
pragma solidity ^0.8.28;

import {IERC20} from "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import {SafeERC20} from "@openzeppelin/contracts/token/ERC20/utils/SafeERC20.sol";
import {ReentrancyGuard} from "@openzeppelin/contracts/utils/ReentrancyGuard.sol";

/// @notice Multi-position linear unlock staking contract.
/// @dev Each stake creates an independent position: t0 is the linear unlock start
time,
/// t1 is the unlock end time, amount is the total locked amount, and claimed is the
total claimed amount.
contract Staking is ReentrancyGuard {
    using SafeERC20 for IERC20;

    struct StakeInfo {
        uint256 t0;
        uint256 t1;
        uint256 amount;
        uint256 claimed;
    }

    struct StakeInfoView {
        uint256 id;
        uint256 t0;
        uint256 t1;
        uint256 amount;
        uint256 claimed;
        uint256 claimable;
    }

    struct WithdrawRequest {
        uint256 id;
        uint256 amount;
    }

    IERC20 public immutable stakingToken;

    mapping(address user => StakeInfo[] stakes) private _stakes;
```

```
event Staked(address indexed user, uint256 indexed id, uint256 t0, uint256 t1,
uint256 amount);
event Withdrawn(address indexed user, uint256 indexed id, uint256 amount);

error InvalidStakingToken();
error InvalidStakeAmount();
error InvalidStakeTime();
error InvalidStakeId();
error InvalidWithdrawAmount();

constructor(address stakingToken_) {
    if (stakingToken_ == address(0)) revert InvalidStakingToken();
    stakingToken = IERC20(stakingToken_);
}

/// @notice Creates a locked staking position. The caller must approve this
contract before staking.
/// @param t0 Linear unlock start timestamp.
/// @param t1 Linear unlock end timestamp. Must be greater than t0.
/// @param amount Total locked amount.
/// @return id The newly created position id for the caller.
function stake(uint256 t0, uint256 t1, uint256 amount) external nonReentrant
returns (uint256 id) {
    if (amount == 0) revert InvalidStakeAmount();
    if (block.timestamp >= t0 || t1 <= t0) revert InvalidStakeTime();

    id = _stakes[msg.sender].length;
    _stakes[msg.sender].push(StakeInfo({t0: t0, t1: t1, amount: amount, claimed:
0}));
    stakingToken.safeTransferFrom(msg.sender, address(this), amount);

    emit Staked(msg.sender, id, t0, t1, amount);
}

/// @notice Withdraws specified currently unlocked amounts for selected positions.
function withdraw(WithdrawRequest[] calldata requests) external nonReentrant {
    StakeInfo[] storage stakes = _stakes[msg.sender];

    for (uint256 i = 0; i < requests.length; ++i) {
        WithdrawRequest calldata request = requests[i];
        if (request.id >= stakes.length) revert InvalidStakeId();

        StakeInfo storage s = stakes[request.id];
        if (request.amount == 0 || request.amount > _claimable(s,
```

```
block.timestamp)) revert InvalidWithdrawAmount();

    s.claimed += request.amount;
    stakingToken.safeTransfer(msg.sender, request.amount);

    emit Withdrawn(msg.sender, request.id, request.amount);
}
}

/// @notice Returns all staking positions for a user with the current claimable
amount.
function info(address user) external view returns (StakeInfoView[] memory rows) {
    StakeInfo[] storage stakes = _stakes[user];
    uint256 length = stakes.length;
    rows = new StakeInfoView[](length);

    for (uint256 i = 0; i < length; ++i) {
        StakeInfo storage s = stakes[i];
        rows[i] = StakeInfoView({
            id: i,
            t0: s.t0,
            t1: s.t1,
            amount: s.amount,
            claimed: s.claimed,
            claimable: _claimable(s, block.timestamp)
        });
    }
}

function _claimable(StakeInfo storage s, uint256 timestamp) private view returns
(uint256) {
    if (timestamp <= s.t0) return 0;
    if (timestamp >= s.t1) return s.amount - s.claimed;
    return (s.amount * (timestamp - s.t0)) / (s.t1 - s.t0) - s.claimed;
}
}
```


Statement

SlowMist issues this report with reference to the facts that have occurred or existed before the issuance of this report, and only assumes corresponding responsibility based on these.

For the facts that occurred or existed after the issuance, SlowMist is not able to judge the security status of this project, and is not responsible for them. The security audit analysis and other contents of this report are based on the documents and materials provided to SlowMist by the information provider till the date of the insurance report (referred to as "provided information"). SlowMist assumes: The information provided is not missing, tampered with, deleted or concealed. If the information provided is missing, tampered with, deleted, concealed, or inconsistent with the actual situation, the SlowMist shall not be liable for any loss or adverse effect resulting therefrom. SlowMist only conducts the agreed security audit on the security situation of the project and issues this report. SlowMist is not responsible for the background and other conditions of the project.



Official Website
www.slowmist.com



E-mail
team@slowmist.com



Twitter
[@SlowMist_Team](https://twitter.com/SlowMist_Team)



Github
<https://github.com/slowmist>